

LINUX SCENARIO BASED Q&A DAY 4

Q1: Port 443 is open, but HTTPS is not working

🧩 Scenario:

The website fails to load over https://, even though port 443 is confirmed to be open.

🧠 Possible Root Cause:

Common causes include:

- Missing or expired SSL/TLS certificate
- Incorrect SSL configuration in the web server (Nginx or Apache)

🔧 Solution:

1. Verify the certificate:

```
openssl s_client -connect yourdomain.com:443
```

Look for certificate details, expiry date, and any errors.

2. Check web server configuration:

- For Nginx: Inspect `ssl_certificate` and `ssl_certificate_key` directives.
- For Apache: Check `SSLCertificateFile` and `SSLCertificateKeyFile`.

3. Renew the certificate (if using Certbot):

```
sudo certbot renew
```

✅ Tip:

Automate SSL renewal using a cron job or systemd timer to avoid downtime due to expired certificates.

Q2: Server Cannot Resolve Internal Hostname

🧩 Scenario:

Running `ping myapp.internal` fails, even though the IP address is correct.

🧠 Possible Root Cause:

- Missing or incorrect entry in `/etc/hosts`
- Misconfigured or broken internal DNS resolver

🔧 Solution:

1. Add a manual entry to `/etc/hosts`:

```
echo "10.0.1.5 myapp.internal" | sudo tee -a /etc/hosts
```

2. Alternatively, fix internal DNS resolution:

- Check `/etc/resolv.conf`
- Validate internal DNS server settings

✅ Tip:

Avoid hardcoding hostnames for dynamic services. Rely on proper internal DNS for scalability and reliability.

Q3: Server Reachable via IP, but Not by Domain Name

🧩 Scenario:

You can successfully ping the server using its IP address, but not using its domain name.

🧠 Possible Root Cause:

DNS resolution is failing, either due to incorrect DNS settings or outdated records.

🔧 Solution:

1. Diagnose with DNS tools:

```
dig yourdomain.com
nslookup yourdomain.com
```

2. Fix the issue:

- Check and update `/etc/resolv.conf` if using a custom resolver
- Ensure your DNS records are correctly configured and propagated

✅ Tip:

If the domain was recently updated, verify that the DNS TTL has expired and changes have propagated fully.

Q4: iptables Rules Lost After Reboot

🧩 Scenario:

Custom firewall (iptables) rules are working initially but disappear after a system reboot.

🧠 Possible Root Cause:

The rules were not saved to persistent storage, so they aren't reloaded at boot time.

🔧 Solution:

1. Make iptables rules persistent:

```
sudo apt install iptables-persistent
sudo netfilter-persistent save
```

2. (Optional): Manually save and restore rules using:

- `iptables-save > /etc/iptables/rules.v4`
- `iptables-restore < /etc/iptables/rules.v4`

✅ Tip:

In production environments, always use `iptables-save` and `iptables-restore` to manage persistent firewall configurations.

Q5: Public IP Changes After Reboot (Cloud Server)

🧩 Scenario:

After rebooting a cloud server, SSH fails because the public IP has changed.

🧠 Possible Root Cause:

The instance was using an ephemeral (dynamic) public IP instead of a static (elastic) one.

🔧 Solution:

In AWS: Allocate an Elastic IP and associate it with the instance

In GCP: Reserve a static external IP and assign it to the VM

✅ Tip:

Always assign a static public IP to production servers to ensure stable connectivity and avoid disruptions after reboot.

Q6: Application Hangs Without Responding, but Still Visible in ps

🧩 Scenario:

The application appears frozen, it doesn't respond to input or produce output, but it's still listed in ps.

🧠 Possible Root Cause:

The process is likely stuck in an uninterruptible sleep (state 'D'), often due to disk I/O wait or hardware issues.

🔧 Solution:

1. Check the process state:

```
ps -eo pid,stat,cmd | grep '^ *PID'
```

Look for processes in state 'D' (Disk Sleep).

2. Investigate disk activity:

```
iostat
```

```
dmesg | tail
```

These can reveal I/O bottlenecks, disk errors, or kernel messages.

✅ Tip:

If a process is blocked in I/O wait, restarting the application (or system) is often the only resolution.

Q7: High Number of Defunct (Zombie) Processes

🧩 Scenario:

ps aux displays many processes in <defunct> status, indicating zombie processes.

🧠 Possible Root Cause:

The parent process is not properly handling the termination of its child processes, it's failing to call wait() to collect their exit status.

 Solution:

1. Identify the parent process:

```
ps -ef | grep defunct
```

2. Restart or signal the parent process:

```
kill -HUP <PARENT_PID>
```

This prompts the parent to reload its configuration or handle children correctly.

 Tip:

Well-designed applications handle child process termination using `wait()` or similar system calls. Continuously monitor and fix poorly behaving processes to avoid resource leaks.

Q8: Process Refuses to Terminate, Even with `kill -9` Scenario:

You attempt to stop a process using `kill <PID>`, and even `kill -9` has no effect; the process remains.

 Possible Root Cause:

The process is stuck in an uninterruptible sleep (state 'D'), typically due to I/O wait or a kernel-level operation (e.g., disk or NFS hang). In this state, the kernel prevents it from being killed.

 Solution:

1. Check process status:

```
ps -o pid,stat,cmd -p <PID>
```

Look for status D (uninterruptible sleep).

2. Resolution: Unfortunately, the only reliable fix is to reboot the system.

 Tip:

This situation is uncommon but often linked to disk, NFS, or hardware issues. After rebooting, investigate and resolve the underlying I/O problem to prevent recurrence.

Q9: Long-Running Command Stopped After Closing Terminal

 Scenario:

You started a long-running task but accidentally closed the terminal; the process was terminated.

 Possible Root Cause:

When the terminal session ends, any foreground processes tied to it are also terminated.

 Solution:

- Use `nohup` to run the command independently of the terminal: `nohup your-command &`
- Or use a terminal multiplexer like `screen` or `tmux` to keep sessions running in the background:
`screen` # or `tmux`

 Tip:

For long-running or production commands, always use `screen` or `tmux` to ensure stability and recovery in case of disconnects.

Q10: CPU Usage at 100%, System Lagging Severely

🧩 Scenario:

The system becomes sluggish or unresponsive. CPU fans spin loudly, and all cores are maxed out.

🧠 Possible Root Cause:

A process (or multiple processes) is consuming excessive CPU, often due to an infinite loop, a runaway thread, or poorly optimized code.

🔧 Solution:

1. Identify high CPU usage:

```
top -o %CPU
```

2. Terminate the offending process:

```
kill -9 <PID>
```

✅ Tip:

To prevent such issues in the future, use tools like `cpulimit` or `nice` to restrict CPU usage of background or non-critical tasks.